Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio.

Equipo BugSlayer

1- René Díaz González C311
2- Liz Cartaya Salabarria C311
3- Daniel A. Estupiñan Reyes C311
4- Gilberto Rodríguez Crespo C311
5- Maya Ramón Castellanos C312
6- David Castillo Rodríguez C311

El orden anterior se corresponde con el número de la pregunta realizada por cada integrante
Integrante 1: pregunta 1.
Integrante 2: pregunta 2.
Integrante 3: pregunta 3.
Integrante 4: pregunta 4.
Integrante 5: pregunta 5.
Integrante 6: pregunta 6.

23/10/2026

`Enlace al Repositorio en GitHub:`
`https://github.com/cartayaliz/travel-itinerary-generator.git`

1 Análisis de Trade-offs (Arquitectura N-Capas vs. Clean/Onion/Hexagonal)

Un equipo de desarrollo está construyendo un sistema de gestión de inventario. Inicialmente, eligen una Arquitectura N-Capas tradicional por su simplicidad y familiaridad. Después de un año, el sistema se ha vuelto un monolito difícil de mantener, donde los cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario, y las pruebas unitarias son lentas y frágiles. Con base en los principios de la Arquitectura Limpia y la Arquitectura Hexagonal, expli- que detalladamente qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adapta- dores (Cockburn) resolverían estructuralmente este problema. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

En una Arquitectura N-Capas tradicional, el error fundamental de acoplamiento que se comete es que las capas superiores como la lógica de negocio y la interfaz de usuario dependen directamente de los detalles de infraestructura, frameworks y persistencia. Aunque conceptualmente las dependencias fluyen de arriba hacia abajo, en la práctica el código del negocio queda atado a esquemas de bases de datos relacionales específicos y tecnologías concretas. Esto genera un monolito rígido y difícil de mantener donde cualquier cambio en la base de datos afecta directamente a la lógica de negocio y a la interfaz, volviendo además las pruebas unitarias lentas y frágiles al requerir entornos e infraestructura reales para ejecutarse.

Para resolver este problema estructuralmente, la Arquitectura Limpia introduce la Regla de Dependencia(Martin, 2017), la cual establece que las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de mayor nivel y las entidades del núcleo de negocio, convirtiendo los detalles en "plugins".

Por su parte, la Arquitectura Hexagonal de Alistair Cockburn aísla la lógica de la aplicación del exterior mediante puertos (interfaces) y adaptadores (implementaciones)(Cockburn, 2005), logrando una simetría que protege el dominio.

En cuanto al impacto en la testabilidad, este cambio es radicalmente positivo: al aplicar la inversión de dependencias, la lógica de negocio puede ser probada de forma totalmente aislada utilizando dobles de prueba, lo que elimina la necesidad de conexiones reales a bases de datos y acelera las pruebas (Martin, 2017). Respecto a la capacidad de cambiar la base de datos de SQL Server a MongoDB, el impacto en la lógica de negocio es nulo. Dado que el núcleo funcional interactúa exclusivamente con abstracciones (puertos), cam-

biar de motor de base de datos no afecta al dominio; únicamente se requiere implementar un nuevo adaptador de persistencia para MongoDB que cumpla con la interfaz definida (Palermo, 2008).

2 Comparación de Estilos: La Simetría del Hexágono vs. la Jerarquía de las Capas

La Arquitectura Hexagonal de Cockburn y la Arquitectura Onion de Palermo comparten el objetivo de aislar el núcleo de negocio, pero utilizan representaciones visuales y metáforas radicalmente diferentes (un hexágono simétrico frente a capas concéntricas).

2.1 Analice en profundidad qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir de forma intuitiva.

En la arquitectura en capas tradicional, el principal inconveniente es el acoplamiento de la interfaz de usuario y la lógica de negocio con el acceso a datos. Esto implica que la interfaz de usuario puede quedar condicionada por la lógica de negocio y que esta, a su vez, puede depender del acceso a datos, ya que las dependencias transitivas siguen siendo dependencias (Palermo, 2008). Como consecuencia, el acoplamiento dificulta actualizar fácilmente las partes del sistema y puede llevar a que este quede obsoleto y posteriormente sea reescrito. En la arquitectura en capas de arriba hacia abajo, cada capa depende de las capas inferiores y, además, suele depender de infraestructura común y servicios de utilidad. La gran desventaja de esta organización es el acoplamiento que crea (Palermo, 2008).

La Arquitectura Onion busca solucionar este problema invirtiendo la dirección de las dependencias, de manera que las capas externas dependan de las internas y el núcleo del dominio no dependa de la infraestructura (Palermo, 2008). Sin embargo, aquí aparece una diferencia importante con la Arquitectura Hexagonal, pues aunque ambas comparten el objetivo de aislar el núcleo del negocio, utilizan representaciones visuales. La Arquitectura Onion utiliza capas concéntricas para mostrar cómo se organizan las dependencias, mientras que Cockburn utiliza un hexágono para representar las distintas formas en que la aplicación puede comunicarse con el exterior (Palermo, 2008; Cockburn, 2005).

La principal diferencia está en que la Arquitectura Hexagonal no establece una dirección privilegiada para las interacciones de la aplicación con el exterior. En las arquitecturas tradicionales, con frecuencia se representa la interfaz de usuario en la parte superior y el acceso a datos en la inferior, lo que puede dar a entender que el sistema funciona siguiendo una dirección determinada

(Palermo, 2008). Por otro lado, Cockburn representa el núcleo de la aplicación rodeado de diferentes puntos de conexión, mostrando que puede interactuar con distintos elementos externos sin que ninguno tenga prioridad por su posición. Por ejemplo, una aplicación puede recibir solicitudes desde una interfaz web, una API o un sistema de pruebas, y comunicarse con una base de datos o con servicios externos. Todos estos mecanismos pueden conectarse con el núcleo sin que la lógica del negocio dependa directamente de sus tecnologías específicas (Cockburn, 2005).

En este sentido, la simetría del hexágono ayuda a evitar la idea de que la aplicación debe organizarse alrededor de una única dirección de interacción, como puede sugerir una representación de capas apiladas. La aplicación puede recibir solicitudes desde distintos puntos y comunicarse con diferentes sistemas externos, sin que alguno de estos tenga prioridad por su posición en el diagrama. Por eso, Cockburn aclara que el hexágono no tiene seis lados por una razón numérica, sino para evitar la asimetría que puede transmitir una representación orientada de arriba hacia abajo o de izquierda a derecha. De esta manera, el hexágono no busca representar un número específico de conexiones, sino destacar que la aplicación puede comunicarse con el exterior desde distintos puntos, sin dar prioridad a uno sobre los demás ni acoplar la lógica de negocio a las tecnologías utilizadas (Cockburn, 2005).

En resumen, mientras que la representación por capas destaca la organización jerárquica de los componentes, la simetría del hexágono resalta que la aplicación puede interactuar con el exterior mediante distintos mecanismos, sin que ninguno tenga prioridad por su posición en el diagrama.

2.2 ¿Cómo se traduce el concepto de Puerto y Adaptador en la práctica de definir interfaces en las capas internas de la Arquitectura Onion?

En la práctica, el concepto de puertos y adaptadores de la Arquitectura Hexagonal también puede aplicarse a la Arquitectura Onion mediante la definición de interfaces en las capas internas (Palermo, 2008; Cockburn, 2005). Por ejemplo, si la lógica de negocio necesita guardar información, puede utilizar una interfaz de repositorio que defina las operaciones necesarias sin depender de una tecnología de base de datos específica. La implementación de esa interfaz se encuentra en una capa externa y se encarga de conectar la aplicación con la tecnología de persistencia utilizada. En este caso, la interfaz actúa como un puerto que define cómo se comunica la aplicación con el exterior, mientras que la implementación concreta funciona como un adaptador (Cockburn, 2005). De esta manera, es posible cambiar la tecnología utilizada sin tener que modificar la lógica de negocio (Palermo, 2008; Cockburn, 2005).

2.3 ¿Por qué Cockburn arma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

Como ya hemos visto antes, Cockburn aclara que el hexágono no tiene seis lados por una razón numérica, sino para evitar la asimetría que pueden transmitir las representaciones tradicionales. En los diagramas de capas apiladas, la posición superior o inferior de los componentes puede sugerir una dirección determinada o dar mayor relevancia visual a ciertos elementos. El hexágono busca eliminar esa orientación y mostrar que la aplicación puede comunicarse con diferentes sistemas externos mediante distintos puntos de conexión, sin que ninguno tenga prioridad por su ubicación en el diagrama. Por tanto, los seis lados son una elección visual y no representan un número fijo de puertos o adaptadores (Cockburn, 2005).

3 Aplicación de DDD y Bounded Context en un Dominio Complejo

Una empresa de seguros necesita desarrollar un sistema para gestionar pólizas de automóviles. El sistema debe interactuar con un departamento de siniestros, un sistema de facturación de primas y una aplicación para que los ajustadores de seguros evalúen daños en el lugar del accidente. Cada uno de estos subsistemas tiene su propia terminología y preocupaciones (ej: "póliza" para un actuario es un contrato con cláusulas, para facturación es una fuente de ingresos, y para un ajustador es un número de referencia). Basado en los conceptos de Domain-Driven Design (Evans) y Bounded Context (Richards & Ford), explique cómo dividiría este sistema en varios contextos delimitados. ¿Cómo evitaría que el modelo de "Póliza" se contamine con detalles de siniestros o facturación? ¿Cómo se comunicarían estos contextos de manera desacoplada? Relacione esta solución con la implementación de una Arquitectura Clean) y la creación de un Adaptador Orientada al Dominio (como Onion o de Servicio para la comunicación externa, tal como se describe en la Guía de Arquitectura N-Capas.

El problema es que "Póliza" significa cosas distintas según el área: para el actuario es un contrato con cláusulas, para facturación es una fuente de ingresos, y para el ajustador es solo un número de referencia. Si hacemos un único modelo de Póliza que sirva para todos, nos queda una clase gigante llena de campos que a cada área le sobran, y encima terminamos metiendo detalles de siniestros en facturación o al revés. Eso es justo lo que Evans (2003) dice que no hay que hacer: un modelo de dominio vale solo dentro de una frontera, el Bounded Context, y el mismo término puede significar cosas distintas en con-

textos distintos sin que sea un error.

La solución no es unificar sino separar. Siguiendo a Richards y Ford (2020), que recomiendan nombrar los contextos por dominio y no por función, dividiría el sistema en cuatro contextos: Suscripción/Actuaría, donde la Póliza es el contrato con sus cláusulas; Siniestros, donde determina si hay cobertura y cuánto se indemniza; Facturación, donde es la fuente de ingresos con primas y estados de pago; y Evaluación de Ajustadores, donde es apenas un número de referencia. Cada contexto tiene su propio modelo y lo único que se comparte es un identificador, tipo PolicyId, nunca la entidad completa. Si Siniestros importara la clase Policy de Suscripción, cualquier cambio en el contrato afectaría directamente al módulo de siniestros, que no tiene por qué saber de cláusulas.

Para que el modelo no se contamine se usa una Capa Anticorrupción (Evans, 2003), que traduce entre contextos. Por ejemplo, si Siniestros necesita saber la cobertura de una póliza, define su propia interfaz PolicyInfoQueryPort que dice "necesito la cobertura de esta póliza", y en infraestructura un adaptador llama al sistema de suscripción y traduce la respuesta a un objeto de valor que Siniestros entienda. Así, si Suscripción cambia su API, solo se toca el adaptador y no el dominio de Siniestros.

Esa comunicación desacoplada es lo que De la Torre Llorente et al. (2010) llaman Adaptadores de Servicio en la Guía de Arquitectura N-Capas, y en la práctica son los mismos adaptadores de la Arquitectura Hexagonal de Cockburn (2005): el dominio define un puerto y la infraestructura implementa el adaptador que se conecta al sistema externo. Es el Principio de Inversión de Dependencias de Martin (2017): los detalles dependen de las abstracciones y no al revés. Si mañana facturación cambia de proveedor o de protocolo, solo se cambia el adaptador y la lógica de negocio de los otros contextos queda intacta.

Dentro de cada contexto se aplica una Arquitectura Onion (Palermo, 2008) o Clean (Martin, 2017), con el núcleo conteniendo entidades, objetos de valor e interfaces, la capa de aplicación coordinando casos de uso, y la infraestructura con controladores, repositorios y adaptadores. La regla de dependencia se respeta: todo apunta hacia adentro. Así el modelo de Póliza de cada contexto queda limpio, sin detalles técnicos ni modelos de otros contextos, y el sistema se vuelve más fácil de mantener y extender.

4 Evaluación de la Inversión de Dependencias en la Práctica

Un equipo de desarrollo está trabajando en un sistema de gestión de pedidos. La lógica de negocio principal se encuentra en una clase OrderProcessor que, para completar su tarea, necesita comunicarse con

un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa: OrderProcessor instancia y llama directamente a la clase AddressValidatorService, que a su vez depende de una librería HTTP específica para comunicarse con el servicio externo. Con base en los principios de la Inversión de Dependencias (Martin) y la Regla de Dependencia (Martin), analice y responda:

4.1 a)¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? Explique específicamente qué dirección deberían tener las dependencias del código fuente y por qué la implementación actual no la cumple.

Violación de los principios arquitectónicos y dirección de dependencias
La dependencia directa entre OrderProcessor y AddressValidatorService viola los principios fundamentales de las arquitecturas Hexagonal (Cockburn, 2005), Limpia (Martin, 2017) y Onion (Palermo, 2008) porque invierte el sentido correcto en que deben fluir las dependencias del código fuente.

Según la Regla de Dependencia de Robert C. Martin (Martin, 2017), las dependencias del código fuente deben apuntar estrictamente hacia adentro, es decir, hacia las políticas de mayor nivel, las entidades y el núcleo de negocio. Los detalles técnicos de infraestructura como bases de datos, frameworks UI y librerías de comunicación HTTP deben ser "plugins" externos que dependen del núcleo, y nunca a la inversa. Al instanciar y llamar directamente a un servicio de infraestructura atado a una librería HTTP externa desde la lógica de negocio (OrderProcessor), se genera un acoplamiento rígido hacia el exterior. Esto provoca que el núcleo del negocio quede contaminado por detalles tecnológicos externos.

4.2 b)Rediseñe la interacción entre OrderProcessor y AddressValidatorService para que se alinee con los principios de estas arquitecturas. Defina claramente qué componente (interfaz) debe residir en el núcleo de la aplicación y qué componente debe residir en la capa de infraestructura o en el exterior del hexágono. Justifique su diseño basándose en el principio de que "los detalles deben depender de las abstracciones".

Rediseño de la interacción y aplicación del principio de abstracciones
Para corregir esta interacción y alinearla con los principios de las arquitecturas orientadas al dominio y limpieza: Debe residir una interfaz por ejemplo, IAddressValidator definida puramente en términos de las necesidades del dominio. La clase OrderProcessor dependerá exclusivamente de esta abstracción. La clase concreta que realiza la llamada HTTP por ejemplo, HttpAddressVa-

lidatorAdapter y la librería HTTP específica residen en la capa externa de infraestructura. Esta clase implementa la interfaz IAddressValidator definida en el núcleo.

Mediante el Principio de Inversión de Dependencias, el control se invierte: aunque el flujo de ejecución en tiempo de ejecución cruza la frontera hacia afuera, las dependencias del código fuente apuntan hacia adentro. La implementación HTTP y el proveedor externo dependen de la abstracción definida en el núcleo, protegiendo a la lógica de negocio de los cambios tecnológicos.

4.3 c) Explique cómo este nuevo diseño permite que la lógica de negocio (OrderProcessor) sea probada de forma aislada, sin necesidad de realizar llamadas HTTP reales al servicio externo, y sin depender de la librería HTTP subyacente. ¿Qué implicaciones tiene este diseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro?

Impacto en la testabilidad, mantenibilidad y flexibilidad futura
Al depender OrderProcessor de una interfaz abstracta IAddressValidator en lugar de una implementación concreta, la lógica de negocio puede probarse de forma totalmente aislada. Durante las pruebas unitarias, es posible inyectar un doble de prueba que implemente la interfaz sin necesidad de realizar solicitudes HTTP reales a servicios externos ni requerir la librería HTTP subyacente. Esto garantiza pruebas rápidas, estables y deterministas.

Este diseño desacoplado incrementa notablemente la mantenibilidad del sistema. Si en el futuro la empresa decide cambiar de proveedor de validación de direcciones o modificar la librería HTTP subyacente, la lógica de negocio principal (OrderProcessor) no sufrirá ninguna afectación ni requerirá modificaciones de código. Únicamente será necesario crear un nuevo adaptador en la capa externa que cumpla con la interfaz del puerto existente, respetando el Principio de Abierto/Cerrado (OCP).

5 Análisis de un Problema Real: El "Kitty Problem" y el Principio de Abierto/Cerrado

En su libro Fundamentals of Software Architecture, Richards y Ford presentan el "Kitty Problem" para ilustrar un fallo común en el diseño de arquitecturas orientadas a servicios. Describa este problema: un requisito transversal (como la entrega de gatitos) afecta a múltiples servicios previamente existentes (TaxiUI, TaxiFinder, TaxiSelector, etc.), requiriendo cambios coordinados en todos ellos. ¿Por qué la solución de dividir el sistema en servicios basados en verbos o

entidades funcionales (como TaxiFinder o TaxiSelector) llevó a este problema? Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio? Proponga una solución utilizando componentes como Kittens y Rides que se añadan como plugins, explicando cómo esto respeta el Principio de Abierto/Cerrado (OCP) y qué papel juegan los Adaptadores en esta extensión.

Richard y Fords, autores de "Fundamentals of Software Architecture" plantearon el conocido "Kitty problem" utilizando de ejemplo un sistema que manejaba una app de viajes de taxi. En este ejemplo nombraban TaxiUI, TaxiFinder, TaxiSelector, TaxiDispatcher, como los componentes de dicho sistema. Pero surge entonces la pregunta de qué sucede cuando se necesita extender dicho sistema para que por ejemplo, el negocio comience a entregar gaticos además de sus funciones ya implementadas. La respuesta es que se necesitaría crear KittenFinder, KittenSelector, o modificar los servicios existentes para que contemplen la posibilidad de los gaticos, lo cual es engorroso e ineficiente a la larga, puesto que un requisito transversal como la entrega de gaticos no debería afectarlos (Richards & Ford, 2020). Lo que llevó a este problema fue dividir el software por lo que hace, en verbos (finder, selector, etc) en lugar de por lo que realmente representa, o sea dominio, entidades. Añadir un nuevo dominio kitten obliga a cambiar la estructura existente, pero manipular esto no cumple con la separación de responsabilidades ya que la parte encargada de los taxis no debería saber nada de kittens.

Ahora, basándose en principios Domain-Driven Design o DDD (Evans, 2003) y la arquitectura Onion (Palermo, 2008), el diseño debería haber sido diferente para que el cambio afectara a un solo componente. No se debe dividir por verbos sino por dominios nombrados por sustantivo, por ejemplo, un contexto para Rides, y un contexto para Kittens (Bounded Context según Richards y Ford). Hay que aislar la lógica del negocio central para que el núcleo del dominio Rides no sepa de cómo se envía o busca un taxi, solo las reglas para un viaje, y luego para kittens solo se crea un nuevo contexto.

El principio Abierto/cerrado (OCP) plantea que el software debe estar abierto a la extensión pero cerrado a la modificación. En este contexto, no se modifica el código de Rides, sino que se extiende. La solución es usar puertos y adaptadores-arquitectura hexagonal. Se crea un núcleo (Core) del negocio con casos de uso como ProcesarViaje, y un Puerto (o sea una interfaz) llamada IDeliveryService. Luego un adaptador para RidesAdapter y otro para KittensAdapter. La app principal, o sea el núcleo, solo conoce el puerto IDeliveryService.

Siguiendo la Arquitectura Limpia popularizada por Robert C. Martin los detalles externos y los nuevos módulos actúan como plugins del núcleo del negocio, respetando la Regla de la Dependencia (las dependencias solo apuntan

hacia dentro). Por eso, al pedir un gatico, el plugin de Kittens se engancha al sistema, y no toca nada relacionado a taxis, y viceversa. Aquí la función de los adaptadores es actuar como traductores. Permiten que el núcleo, que solo habla el idioma del IDeliveryService o "Necesito entregar algo", se comunique con el exterior, que en este caso sería por ejemplo la base de datos de taxis, la base de datos de gaticos, o la API de pagos. De esa forma ambos comparten sus funciones relacionadas, es decir el viaje, pero mantienen completa separación al ser dominios diferentes. El núcleo solo sabe que se necesita un viaje y toda la lógica relacionada, no para qué es, esto lo define el propio KittensAdapter y RidesAdapter. Por tanto se cumple OCP y el sistema no cae en el problema inicial planteado por Richard y Ford.

En resumen, el código de la UI y la base de datos deben depender del núcleo, nunca al revés, y DDD divide el sistema en contextos delimitados que se comunican mediante eventos o interfaces sin compartir base de datos para que su mejor aislamiento garantice la extensibilidad.

6 Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad

La Arquitectura N-Capas es criticada por su bajo rendimiento en características como "Agilidad general", "Facilidad de despliegue" y "Escalabilidad" (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (open layers) y técnicas como el "fast-lane reader", se pueden mitigar estos problemas.

6.1 Analice esta afirmación. ¿Cómo afecta la introducción de capas abiertas al principio de Aislamiento de Capas y a la mantenibilidad a largo plazo?

El principio de Aislamiento de Capas consiste en que cada capa es independiente de las otras. Asegura que los cambios en los componentes de una capa solo afecten a los de su misma (Richards, 2015). Luego, que una esté marcada como abierta significa que pedidos de capas superiores pueden saltarse esta e ir directo a la inferior (Richards, 2015).

Si se falla en definir qué capas son abiertas o cerradas resulta en contradecir el principio dado, porque si se permite que varias capas superiores tengan acceso directo a capas muy inferiores, usualmente resulta en que modificaciones a los componentes de capas inferiores afectan a los de su capa y superiores (Richards, 2015).

En consecuencia, se pierden los beneficios de la Arquitectura N-Capas como fácil desarrollo y realización de pruebas, y la mantenibilidad a largo plazo ya

que se vuelve difícil y costoso hacer cambios (Richards, 2015; Martin, 2017).

6.2 ¿Por qué la solución de capas abiertas puede llevar a un "sinkhole anti-pattern" y qué dice la regla del 80/20 sobre esto?

Sinkhole anti-pattern describe la situación donde el flujo de pedidos a través de varias capas de la arquitectura son procesos con poca o sin lógica realizada dentro de cada capa (Richards, 2015).

La regla 80/20 es una práctica en donde es típico tener alrededor de un 20% de los pedidos como simples procesos de paso y un 80% de los pedidos teniendo algún tipo de lógica asociada al pedido. Sin embargo, si la proporción está invertida y la mayoría de los procesos son simples procesos de paso, se considera selectivamente abrir capas para mitigar, teniendo en cuenta que si no se controla provocarán el efecto contrario porque ninguna de esas capas intermedias se usan y no tienen propósito (Richards, 2015).

6.3 Explique por qué, para un sistema con requisitos de alta escalabilidad y agilidad, una arquitectura como la Hexagonal o la Limpia, que se basan en los principios de DDD y desacoplamiento, son inherentemente más adecuadas que una Arquitectura N-Capas, incluso con optimizaciones.

Aunque en la Arquitectura N-Capas los cambios son aislados por el principio de Aislamiento de Capas, aún es trabajoso y costoso hacer cambios en este patrón por la tendencia monolítica de la mayoría de implementaciones y los componentes estrechamente relacionados que trae el patrón (Richards, 2015). Las mismas razones llevan a que el proyecto sea costoso de escalar, ya sea separando en más capas o replicando la aplicación dentro de varios nodos, lo cual provoca que los componentes sean relativamente más grandes (Richards, 2015).

La Arquitectura Hexagonal y Limpia son más adecuadas porque sus patrones protegen a la infraestructura, la lógica de negocio y el dominio (Cockburn, 2005; Martin, 2017).

- Enfoque Hexagonal:
La arquitectura resuelve estos problemas separando la aplicación (interior) que se comunica mediante unos puertos de entrada y salida (lados) con componentes ajenos (exterior) sin saber que estos son tratados de manera simétrica usando adaptadores. Hace énfasis en la asimetría interior-exterior y la similitud de los puertos, alejándose del modelo unidimensional de la Arquitectura N-Capas (Cockburn, 2005).

- Enfoque Limpia:
Desarrolla una teoría basándose en arquitecturas como la hexagonal estableciendo capas concéntricas que separan al menos lógica de negocio, interfaces y componentes externos (UI, bases de datos o pruebas). También estas capas son independientes de cualquier software externo a la aplicación, comunicándose con ellos a través de interfaces, y siguen la Regla de Dependecia: "Dependencias de código fuente deben apuntar hacia dentro, a políticas de mayor nivel" (Martin, 2017).

Construyendo software siguiendo las reglas de la Arquitectura Limpia o siguiendo el patrón de puertos-adaptadores de la Arquitectura Hexagonal no es complicado y evita problemas de mantenimiento y aislamiento de la aplicación. También son métodos que permiten usar los componentes externos como herramientas sin depender de ellos y realizar pruebas sobre el código sencillamente en una estructura que no es difícil de escalar a diferencia de la Arquitectura N-Capas (Cockburn, 2005; Richards, 2015; Martin, 2017).

7 Bibliografía

- Cockburn, A. (2005). Hexagonal Architecture (Ports and Adapters). Recuperado de https://alistair.cockburn.us/hexagonal-architecture/

- De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., Calvarro Nelson, J. (2010). Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador). Microsoft Ibérica.

- Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

- Palermo, J. (2008). The Onion Architecture. Recuperado de https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

- Richards, M., Ford, N. (2020). Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media.

8 Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario se utilizó ChatGPT (GPT-6), Deepseek (6.2.3) y Gemini (flash 3) como herramientas de apoyo para revisar la redacción y aclarar conceptos de arquitectura de software. Las sugerencias se utilizaron como apoyo para revisar y mejorar el texto, sin sustituir la consulta de las fuentes originales. Los integrantes responsables de estas consultas revisaron y adaptaron las propuestas utilizadas en el documento. El equipo asume la responsabilidad final de verificar la exactitud de la información, la adecuación de las referencias y el contenido completo del trabajo.